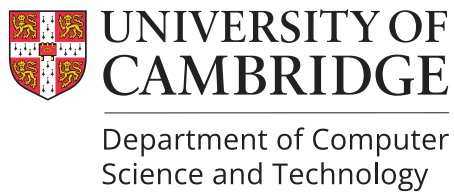


Categorical Imperative Programming

Type theory, refinement, and semantics for SSA

Jad-Elkhaleq Ghalayini

March 2026



This thesis is the result of my own work and includes nothing which is the outcome of work done in collaboration except as declared in the preface and specified in the text. It is not substantially the same as any work that has already been submitted, or is being concurrently submitted, for any degree, diploma or other qualification at the University of Cambridge or any other University or similar institution except as declared in the preface and specified in the text. It does not exceed the prescribed word limit for the relevant Degree Committee.

Contents

1. Introduction	4
2. Type-Theoretic SSA	4
2.1. Basic Arithmetic	7
2.1.1. State	10
Bibliography	11
1. Sample Appendix	12

1. Introduction

The fundamental question of this thesis is: *what does a computer program mean?*

We can approach this in two ways: from the bottom-up or the top-down.

The former tends to be more natural for computer scientists.

A computer is a machine for following instructions – a program is a sequence of instructions for it to follow.

In particular, a computer has an *instruction set* – a finite set of *operations* which it can perform, which read from and write to its *state* – that is, the current state of the registers, program counters, memory, interrupts, and all the other bits which make a modern CPU tick.

Which is a *lot*.

This thesis is about the *denotational semantics* of the *static single assignment* – or *SSA* – compiler *intermediate representation* – or *IR*.

I want to start by justifying this choice of topic.

- *Why* do we need a semantics for a *compiler IR*?

Because high-level languages have too many features to effectively study *or* to lower in one pass.

Because low-level languages are designed to be *executed* efficiently – making them difficult both to reason about *or* to lower in one pass.

- *Why* should we use *SSA* as our compiler *IR*?

Practically: most modern compilers use it.

Theoretically: it's right at the intersection of functional and imperative programming – letting us use it as a *thin waist* to develop the *isomorphism* between the two.

- *Why* should we give a *denotational* semantics?

Because we want to reason about programs *algebraically* – and to do so in a *compositional* manner.

2. Type-Theoretic SSA

Premature optimization is the root of all evil.

— Donald Knuth

The goal of this chapter is to give a formal presentation of *SSA* – fully specifying its *syntax*, *type theory*, and *semantics*. Rather than do this all at once, we're instead going to “build up to” *SSA* by studying a sequence of increasingly complex languages, each of which embeds into the next.

In particular, we'll begin by introducing:

1. The language of simple arithmetic expressions, λ_{arith} , which we'll extend to:
2. The language of *effectful* expressions, λ_{expr} , which we'll restrict to:
3. The language of *operations*, λ_{op} – a single effectful expression which depends on variables and constants. This will allow us to define

4. The language of *basic blocks*, λ_{bb} , – straight-line sequences of operations – which we’ll show *equivalent* to the language of effectful expressions with *let-bindings*. By adding conditional branches, we can then recover
5. The language of *acyclic regions*, λ_{areg} , – single-entry, single-exit *acyclic* control-flow graphs of basic blocks – which we’ll show *equivalent* to the language of effectful expressions with *case-expressions*

Finally, we’ll drop the acyclicity restriction to get to our final type-theoretic presentation of SSA:

6. The language of *regions*, λ_{reg} , – single-entry, multiple-exit control-flow graphs of basic blocks – which we’ll show *equivalent* to the language of effectful expressions with *iteration*

Our goal is to give a precise mathematical description of each of these languages – consequently, each language’s section will follow a somewhat repetitive pattern, where we specify:

- A *syntax*, which specifies how to tell whether a term is *grammatical* – like “ $x * (y + z)$ ” – or nonsense – like “ $C + +$ ”.
- A *type system*, which consists of:
 - A collection of ways we can *interpret* a given term (natural number, string, graph, tensor, neural network...) – that is, a set of *types* A, B, C
 - A description of the *contexts* in which we can interpret terms
 - A *typing relation* – which tells us in which contexts a given term can be interpreted in a given way.

Concretely: we might be able to interpret $x + x$ as an integer in the context $x : \mathbb{Z}$, or as a string in the context $x : \text{str}$ – but $x + x$ doesn’t make sense for $x : \text{POBox}$.

Normally, a *type theory* consists of a type system together with an *equational theory*: a relation which tells us when two terms are “the same” in a particular context – for example, we might have $x + y \equiv y + x$ in the context $x : \mathbb{Z}, y : \mathbb{Z}$, but not in the context $x : \text{str}, y : \text{str}$.

This can let us reason about programs, but also *optimize* them – for example, we might rewrite

$$(x + 5) + (x + 5) + (x + 5) \quad \longrightarrow \quad \text{let } y = x + 5; 3 \cdot y$$

Since our main focus will be the study of compiler optimizations, we will hence generalize equational theories to *refinement theories*: where a refinement theory is a relation which tells us that replacing a term t with another term t' is always sound, i.e., that t' *refines* t – written $t \rightarrow t'$.

We can recover an equational theory by saying that t and t' are *equivalent* – written $t \approx t'$ – whenever $t \rightarrow t'$ and $t' \rightarrow t$. Likewise, every equational theory induces a refinement theory by simply defining $(t \rightarrow t') := (t \approx t')$.

As a concrete example, let’s assume that the result of $x/0$ is *implementation-defined* – and, in particular, may return a different result each time it’s evaluated.

Then it’s sound to rewrite

$$(x/y) + (x/y) \quad \longrightarrow \quad \text{let } z = x/y; z + z$$

since, whether or not $y = 0$, every valid result for the original program is also a valid result for the rewritten program.

On the other hand, for $x, y : \mathbb{Z}$, the rewrite

$$\text{let } z = x/y; 2 \cdot z \not\rightarrow (x/y) + (x/y)$$

would be unsound, since every valid result for the left-hand side is *even* (since it can be written $2 \cdot z$ for some z) but the right-hand side can be *odd* – for example, if $y = 0$, we could have the reduction sequence

$$(x/y) + (x/y) \rightarrow 1 + (x/y) \rightarrow 1 + 0 \rightarrow 1$$

Just like equations, refinements can be *context-dependent* – for example, depending on our semantics, we may have that

$$\text{let } z = x/y; 2 \cdot z \rightarrow (x/y) + (x/y)$$

is a perfectly valid optimization for $x, y : \mathbb{R}$.

At this point, we’ve got a *theory* for our programming language:

- A *syntax*, which sets the domain of discourse
- A *type theory*, which tells us what rules we can use to reason about sentences in our syntax

What we now need to confirm is whether this theory is a *good* one – that is, whether it captures our intuitions about how programs should behave.

To do this, we need to give our theory a *semantics*. There are two primary approaches we might take:

- We can provide an *operational semantics*: a relation which specifies the behaviour of a program by describing how it would be executed by an *abstract machine*.

Our semantics is then *sound* if $t \rightarrow t'$, implies that valid execution of t' is a valid execution of t .

Alternatively,

- We can provide a *denotational semantics*: a mapping from well-formed terms in our syntax to mathematical objects.

Soundness in this case means that every sentence which is true according to our type theory is true when interpreted as a statement about the semantics.

For example, if we interpret programs like $x/(y + z)$ as functions from *valuations* $\text{Var} \rightarrow \mathbb{Z}$ to sets of possible results $\mathcal{P}(\mathbb{Z})$, then we might say our type theory is *sound* if, for all well-formed t, t' ,

$$t \rightarrow t' \implies \llbracket t \rrbracket \subseteq \llbracket t' \rrbracket \quad (\text{pointwise})$$

The standard approach for reasoning about SSA – e.g. in **TODO 1: VELVVM, etc** – is operational. One of the primary contributions of this thesis is to give a *compositional* denotational semantics for SSA – that is, a denotational semantics in which the semantics of a program as a whole is a function of the semantics of its parts. In particular, this will be enabled by *categorical semantics*: where types and contexts are interpreted as *objects* in a *category*.

While the resulting semantics enables compositional reasoning about SSA programs, it is also quite abstract – hence, to validate that it is in fact the “right” semantics, we’ll show that:

- It is *complete* with respect to our refinement theory: that is, $t \rightarrow t'$ if and only if $\llbracket t \rrbracket_{\mathcal{M}} \leq \llbracket t' \rrbracket_{\mathcal{M}}$ for every model $\mathcal{M}$
- Our refinement theory is *sound* with respect to a reasonable operational semantics – and hence our categorical semantics at least *abstracts* the operational semantics.

Let's get started!

2.1. Basic Arithmetic

We'll do it tomorrow

– TODO

We'll start by giving a formal account of the simplest practical language – that of *first-order* arithmetic expressions, like $x + y$ and $a + (5 \cdot b) + \sin(3)$.

Rather than complicate our language by introducing binary operators (like $x + y$), unary operators (like $-x$), and n -ary function applications (like $f(x, y, z)$), we can instead normalize everything to *S-expressions* – or *sexprs* – of the form $(f\ e_1 \dots e_n)$. We can then uniformly write:

- binary operations like $x + y$ as binary sexprs $(+ x\ y)$,
- unary operations $-x$ as unary sexprs $(- x)$,
- function applications $f(x, y, z)$ as n -ary sexprs $(f\ x\ y\ z)$.

In particular, the *operator* f always comes first, and there is no order-of-operations – all bracketing is explicit. Restricting ourselves to *first-order* sexprs means that we always require operators f to be atomic symbols drawn from a fixed set O – in particular, we don't support *partial application*, like $((\text{add } 2)\ 3)$, since that would require $f = (\text{add } 2)$ to be a compound expression.

Formally, we define our *syntax* as follows:

Definition 2.1.1 (First-order S-expressions): fixing

- a set of *variables* $x \in N$
- a set of *operations* $f \in O$
- a set of *constants* $c \in V$

we define the set of *first-order S-expressions* $e \in \text{STm}_N(O)$ to be given by the grammar

$$e ::= x \mid c \mid (f\ e^*)$$

We say a sexpr is *closed* if it contains no free variables – otherwise, it is *open* – where the *free variables* $\text{use}(e)$ of a sexpr e are defined as follows:

$$\text{use}(x) = \{x\} \qquad \text{use}(c) = \{\} \qquad \text{use}((f\ e_1 \dots e_n)) = \bigcup_i \text{use}(e_i)$$

We'll write the set of closed sexprs as $\text{STm}_\emptyset(O)$.

It's easy enough, in the language of simple arithmetic, to define what a closed sexpr “means”, since we can just *evaluate* it to find out.

For example, we might have been taught to evaluate expressions from the “inside out”, like so:

$$(+\ 5\ (\cdot\ 3\ 7)) = (+\ 5\ 21) = 26$$

We can formalize this intuition in terms of a *small-step operational semantics*: given, for each operation $f \in O$, a (partial) function $\text{ev}(f) : V^* \rightarrow V$ telling us what value it returns when called with arguments $c_1, \dots, c_n$, we may define a *reduction relation* $e \rightarrow e'$ on sexprs as follows:

$$\text{step} \frac{\text{ev}(f)(c_1, \dots, c_n) = c}{(f \ c_1 \ \dots \ c_n) \rightarrow c} \quad \text{arg} \frac{c_1, \dots, c_n \in V \quad e_1 \rightarrow e'_1}{(f \ c_1 \ \dots \ c_n \ e_1 \ \dots \ e_m) \rightarrow (f \ c_1 \ \dots \ c_n \ e'_1 \ \dots \ e_m)}$$

Partiality here means that we don't need to assign a meaning to ill-defined expressions like $(/ \ 1 \ 0)$ or $(+)$ (when forced, 0 is a decent choice for both of these) – we can simply leave them as undefined.

Notice that we've fixed the *evaluation order* of sexprs to be left-to-right – that is; we have to finish evaluating all terms to the *left* of a given term before we can get to reducing it.

We say a term e *evaluates* to a constant c – written $e \Downarrow c$ – if there exists a finite sequence of reductions

$$e \rightarrow e_1 \rightarrow \dots \rightarrow e_n \rightarrow c$$

i.e., if $e \rightarrow^* c$ – where R^* denotes the *reflexive, transitive closure* of a relation R .

We call a relation $e \Downarrow c$ a *big-step operational semantics* – as our original intuition for evaluation would suggest, c is a pretty good candidate for the “meaning” of a program $e \Downarrow c$.

In particular, we may define a partial function $\text{ev}(e)$ on closed terms by induction as follows:

$$\text{ev}(c) = c \quad \text{ev}((f \ e_1 \ \dots \ e_n)) = \text{ev}(f)(\text{ev}(e_1), \dots, \text{ev}(e_n))$$

It is straightforward to show that $e \Downarrow \text{ev}(e)$ if and only if $\text{ev}(e)$ is defined.

TODO 2: Consider alternate example: $(\cdot \ (+ \ 1 \ 1) \ x)$

- **Reduces to $(\cdot \ 2 \ x)$ – so these should be “the same”**
- **Intuitively the same as $(+ \ x \ x)$**
- **Intuitively different from $(\cdot \ 3 \ x)$**

We still, however, don't know what an *open* term means: if we try to evaluate

$$(+ \ (+ \ 3 \ 2) \ (\cdot \ x \ 0)) \rightarrow (+ \ 5 \ (\cdot \ x \ 0)) \rightarrow ???$$

the first time we encounter a variable, the relation defined above becomes *stuck* – that is, we reach a term e which is not a constant, and yet does not reduce to anything – so, according to our current conventions, is ill-defined like $(/ \ 1 \ 0)$.

If we fire up a Python REPL and try it, it seems to agree:

```
>>> 5 + (x * 0)
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
5 + (x * 0)
```

```
^
```

```
NameError: name 'x' is not defined
```

On the other hand, if x is in context – say $x = 7$ – this evaluates to 5 as we'd expect:

```
>>> x = 7
```

```
>>> 5 + (0 * x)
```

```
5
```

What this hints at is that the operational semantics of an open term e depends on the context in which it is evaluated – here, on the values of its free variables.

We can model this by defining a *contextual reduction relation* $\gamma \vdash e \rightarrow e'$ parametrized by an *evaluation context* γ – a finitely supported function $\gamma : N \rightarrow V$ assigning a values to variables – as follows:

$$\begin{array}{c} \text{var} \frac{\gamma(x) = c}{\gamma \vdash x \rightarrow c} \qquad \text{step} \frac{\text{ev}(f)(c_1, \dots, c_n) = c}{\gamma \vdash (f \ c_1 \dots c_n) \rightarrow c} \\ \\ \text{arg} \frac{c_1, \dots, c_n \in V \qquad \gamma \vdash e_1 \rightarrow e'_1}{\gamma \vdash (f \ c_1 \dots c_n \ e_1 \dots e_m) \rightarrow (f \ c_1 \dots c_n \ e'_1 \dots e_m)} \end{array}$$

We can then evaluate

$$(x \mapsto 7) \vdash (+ \ (+ \ 3 \ 2) \ (\cdot \ x \ 0)) \rightarrow (+ \ 5 \ (\cdot \ x \ 0)) \rightarrow (+ \ 5 \ (\cdot \ 7 \ 0)) \rightarrow (+ \ 5 \ 0) \rightarrow 5$$

TODO 3: substitution maps open terms to closed terms

TODO 4: discuss:

- algebra: “equivalence under substitution”
- programming: “observational equivalence”

Given a set of *types* $A \in \mathcal{T}$, we’ll define a *stack-typing relation* on O to be any relation

$$f : [A_1, \dots, A_m] \rightarrow [B_1, \dots, B_n]$$

where

- $A_1, \dots, A_m \in \mathcal{T}$ are types – m is called the *input arity* or *arity* of f .
- $B_1, \dots, B_n \in \mathcal{T}$ are types – n is called the *output arity* of f .

For now, we’ll only consider operations with an output arity of 1.

We define a (*typing*) *context* $\Gamma \in \text{Ctx}_N(\mathcal{T})$ to be given by a list of *bindings*

$$x_1 : A_1, \dots, x_n : A_n$$

We can define a simple type system for S-expressions by giving a typing relation $\Gamma \vdash e : A$ that says “under the assumptions of Γ , the S-expression e has type A ”. The rules are:

TODO 5: Typing rules for constants – as distinct from nullary ops

$$\begin{array}{c} \text{var-head} \frac{}{\Gamma, x : A \vdash x : A} \qquad \text{var-tail} \frac{\Gamma \vdash y : B, x \neq y}{\Gamma, x : A \vdash y : B} \\ \\ \text{app} \frac{f : [A_1, \dots, A_n] \rightarrow [B] \quad \forall i. \Gamma \vdash a_i : A_i}{\Gamma \vdash (f \ a_1 \dots a_n) : B} \end{array}$$

TODO 6: Soundness of typing: preservation of typing

TODO 7: Completeness of typing: in the presence of simple types

TODO 8: Church vs. Curry – we will only consider well-typed terms

TODO 9: Doesn’t lose generality – untyping works in this framework due to multi-arity

TODO 10: Substitution and observational equivalence – typing rules for substitution – two open programs are equivalent iff equivalent under every substitution – this is usually where *metavariables* come in

TODO 11: Purity – required for substitution to preserve equivalence – typing rules for purity

TODO 12: Denotational semantics of S-expressions: partial functions

TODO 13: Denotational semantics of S-expressions: *soundness* and *completeness*

TODO 14: Equational theory of S-expressions: *purity* – as otherwise you can't *introduce* divisions

TODO 15: Equational theory of S-expressions: *soundness* and *completeness*

2.1.1. State

TODO 16: Operational semantics of S-expressions – *state*

TODO 17: *Observational equivalence* in the presence of state

TODO 18: Denotational semantics of S-expressions – the partial state monad

TODO 19: Denotational semantics of S-expressions: *soundness* and *completeness*

TODO 20: Equational theory of S-expressions: *purity*

TODO 21: Equational theory of S-expressions: *soundness* and *completeness*

Bibliography

1. Sample Appendix

SAMPLE TEXT